

Problem 1

Class **Node** describes a node of singly-linked list and contains data of type **int** and the reference to the next node of the list.

Write a program operating on lists represented by references to objects of type **Node** which are their heads. Define the following functions:

1. **static Node arrayToList(int[] arr)**
which takes an array of **ints**. The function creates the list of nodes containing integers from the array (in the same order) and returns the head of this list.
2. **static Node[] extract(Node head)**
which takes the reference to the head of a list. The function splits the list into two — one containing only nodes with even data and the other with nodes containing odd data. Two-element array of heads of these lists is returned. NOTE: the **extract** function operates on existing nodes only; no new nodes are created!
3. **static void showList(Node head)**
which prints the list represented by its head.

The following fragment:

```
int[] tab = { 2, 1, 4, 3, 6, 5, 7, 8 };
Node head = arrayToList(tab);
showList(head);
Node[] nodes = extract(head);
showList(nodes[0]);
showList(nodes[1]);
```

should print something like

```
2 1 4 3 6 5 7 8
2 4 6 8
1 3 5 7
```

Do **not** use any classes from the **java.util** package.

Problem 2

Create class **SortedSLL** (*sorted singly-linked list*) implementing singly-linked list of **ints**. The elements of the list are represented by objects of class **Node** containing data (an **int**) and the reference to the next such node. The **SortedSLL** class contains only one method adding elements to the list, **addSorted**, which adds them in such a way that the nodes are always ordered by values of data that they contain in ascending order. Add a method to print the list, so the following fragment

```
SortedSLL list = new SortedSLL();
list.addSorted(4);
list.addSorted(1);
list.addSorted(6);
list.addSorted(3);
list.show();
```

will print

```
[ 1 3 4 6 ]
```

Problem 3

Write a program which checks correctness of brackets (round, curly and square) in a text file (or just a multi-line string). This can be done by reading the text character by character and:

- if an opening bracket has been encountered, put information on its kind (round, curly or square) onto a stack;
- if a closing bracket has been encountered (not necessarily in the same line as the corresponding opening bracket), pop information on the kind of the last opening bracket off the stack and check if it matches the kind of the closing bracket encountered.

Implement the stack by creating class **Node**, as in implementation of singly-linked lists. Objects of type **Node** represent data pushed onto the stack (kinds of brackets, as a value of type **char**, **String**, **int** ...) and contain a reference **next** to the next node, as it is usual in singly-linked lists. Note that stack is a singly-linked list where adding and removing elements is always performed at the beginning, so traversing the list is never needed.

The program reads the input and prints "OK" if all brackets are correct. If an error is detected, the program describes it, prints line number and the offending line itself indicating the location of the error by ^ character under the incorrect bracket. After reading the whole input, the program checks if the stack is empty; if not, it prints information about that and a list of all brackets that have not been closed.

For example, for a file

```
Warsaw(){
London[xxxx] (
Madrid Paris)}
Berlin
```

the program should print something like

```
London[xxxx] (
      ^
ERROR in line 2. '}' found, but ']' expected.
```
